

async

Async JavaScript — The Complete Guide

Callbacks, Promises, async/await — Evolution, Internals, Production Patterns

Prepared by Claude for Akshay

1. Why Async Exists

JavaScript is **single-threaded**. Without async, any slow operation freezes everything. Async means: "Start this task, continue running other code, come back when the result is ready."

```
// Without async: SEQUENTIAL, each waits (3.5s total, page frozen)
const users = fetch("/api/users");      // 2s frozen...
const posts = fetch("/api/posts");      // 1.5s frozen...
// With async: PARALLEL, fires both at once (2s total, not frozen!)
fetch("/api/users").then(data => console.log(data));
fetch("/api/posts").then(data => console.log(data));
console.log("Not frozen!"); // runs immediately
```

■ The Evolution

Era 1: Callbacks (original) — function passed as argument, called when done

Era 2: Promises (ES6 / 2015) — object representing a future value

Era 3: async/await (ES8 / 2017) — syntactic sugar over Promises, looks synchronous

2. Callbacks — The Original Way

A callback is a function you pass to another function: "call me when you're done."

```
function fetchUser(userId, callback) {
  setTimeout(() => {
    const user = { id: userId, name: "Akshay" };
    callback(null, user); // convention: (error, result)
  }, 1000);
}
// Usage
fetchUser("u123", (error, user) => {
  if (error) { console.error(error); return; }
  console.log(user); // { id: "u123", name: "Akshay" }
});
```

Error-First Convention (Node.js): Callback always receives (error, result) — error is always first.

```
fs.readFile("/data.json", "utf8", (err, data) => {
  if (err) { console.error("Failed:", err); return; }
  console.log(data);
});
```

■ Callback Hell (Pyramid of Doom)

When you need results from one callback to start the next, nesting becomes deeply indented and unreadable:

```
getUser(id, (err, user) => {
  if (err) return handleError(err);
  getOrders(user.id, (err, orders) => {
    if (err) return handleError(err);
    getDetails(orders[0].id, (err, details) => {
      if (err) return handleError(err);
      getShipping(details.shipId, (err, shipping) => {
        if (err) return handleError(err);
        console.log(shipping);
        // 4 levels deep! Error handling repeated everywhere.
      });
    });
  });
});
```

Problems: Deeply nested, error handling at every level, hard to parallelize, can't use try/catch, no standard API.

3. Promises — The Modern Foundation

A Promise is an **object representing a future value**. Three states:

```
+-----+ resolve(value) +-----+
| PENDING | -----> | FULFILLED | (.then() runs)
| (waiting)|         | (success) |
+-----+         +-----+
|
| reject(error)      +-----+
+-----> | REJECTED | (.catch() runs)
          | (failure) |
          +-----+

// Once settled (fulfilled/rejected), NEVER changes state again.
```

3.1 Creating a Promise

```
const promise = new Promise((resolve, reject) => {
  // Executor runs IMMEDIATELY (synchronous!)
  if (success) {
    resolve("Data here"); // -> FULFILLED
  } else {
    reject(new Error("Failed")); // -> REJECTED
  }
});
// Consuming
promise
  .then(data => console.log(data)) // if fulfilled
  .catch(err => console.error(err)) // if rejected
  .finally(() => console.log("Done")); // always
```

★ Critical Timing

The executor function inside `new Promise()` runs **synchronously**. But `.then()` / `.catch()` callbacks are always **asynchronous** (Microtask Queue).

```
console.log("1");
const p = new Promise((resolve) => {
  console.log("2"); // SYNC! runs during creation
  resolve("3");
});
p.then(val => console.log(val)); // ASYNC! Microtask Queue
console.log("4");
// Output: 1, 2, 4, 3
// "2" = sync (executor), "3" = async (microtask after sync done)
```

3.2 Promise Chaining — Solving Callback Hell

```
// Callback Hell -> flat Promise chain:
getUser(id)
  .then(user => getOrders(user.id))
  .then(orders => getDetails(orders[0].id))
  .then(details => getShipping(details.shipId))
  .then(shipping => console.log(shipping))
  .catch(err => console.error("Any step failed:", err));
// ONE catch handles errors from ANY step!
```

How chaining works: Each `.then()` returns a **NEW Promise**. Your return value becomes the next Promise's resolved value. If you return a Promise, the chain waits for it.

```
Promise.resolve(1)
  .then(x => x + 1)      // returns 2 -> new Promise(2)
  .then(x => x * 3)      // returns 6 -> new Promise(6)
  .then(x => console.log(x)); // 6
// Returning a Promise: chain WAITS for it
Promise.resolve("start")
  .then(val => new Promise(resolve => {
    setTimeout(() => resolve("delayed"), 1000);
  }))
  .then(val => console.log(val)); // "delayed" (after 1s)
```

3.3 Error Handling in Promises

```
fetch("/api/users")
  .then(res => {
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return res.json();
  })
  .then(data => processData(data))
  .then(result => displayResult(result))
  .catch(err => {
    // Catches error from ANY step above!
    // Errors propagate DOWN the chain until caught
    console.error("Failed:", err.message);
  })
  .finally(() => hideSpinner()); // Always runs
```

■ Unhandled Promise Rejections

A Promise without `.catch()` = unhandled rejection. In Node.js, this can **crash your server!** Always add `.catch()` or use `try/catch` with `await`.

```
// Global handlers for safety net:
// Browser:
window.addEventListener("unhandledrejection", e => {
  console.error("Unhandled:", e.reason);
});
// Node.js:
process.on("unhandledRejection", (reason) => {
  console.error("Unhandled:", reason);
});
```

4. Promise Static Methods — The Power Tools

4.1 Promise.all() — All Must Succeed

```
// Runs ALL in parallel. Resolves when ALL resolve.
// If ANY rejects, whole thing rejects immediately.
const [users, posts, comments] = await Promise.all([
  fetch("/api/users").then(r => r.json()),
  fetch("/api/posts").then(r => r.json()),
  fetch("/api/comments").then(r => r.json()),
]);
// Total time = slowest request (2s), not sum (3.5s)!
// DANGER: one failure loses ALL results
Promise.all([
  Promise.resolve("ok"),
```

```

    Promise.reject("FAIL"),    // -> everything fails!
    Promise.resolve("ok"),
  ]).catch(err => console.log(err)); // "FAIL"

```

4.2 Promise.allSettled() — Get All Results

```

// Waits for ALL to settle. NEVER rejects.
const results = await Promise.allSettled([
  fetch("/api/users").then(r => r.json()),
  fetch("/api/failing"),
  fetch("/api/posts").then(r => r.json()),
]);
// results = [
//   { status: "fulfilled", value: [...users] },
//   { status: "rejected", reason: Error },
//   { status: "fulfilled", value: [...posts] },
// ]
const ok = results.filter(r => r.status === "fulfilled");
const bad = results.filter(r => r.status === "rejected");

```

4.3 Promise.race() — First One Wins

```

// Resolves/rejects with FIRST promise to settle
// Use case: timeout for slow requests
function fetchWithTimeout(url, ms) {
  return Promise.race([
    fetch(url),
    new Promise( (_, reject) =>
      setTimeout(() => reject(new Error("Timeout")), ms)
    ),
  ]);
}
const res = await fetchWithTimeout("/api/slow", 3000);

```

4.4 Promise.any() — First SUCCESS Wins

```

// Resolves with FIRST to FULFILL (ignores rejections)
// Only rejects if ALL fail (AggregateError)
const data = await Promise.any([
  fetch("https://cdn1.example.com/data.json"),
  fetch("https://cdn2.example.com/data.json"),
  fetch("https://cdn3.example.com/data.json"),
]); // Uses whichever CDN responds first!

```

Comparison:

Method	Resolves When	Rejects When	Use Case
Promise.all()	ALL succeed	FIRST fails	Parallel tasks, all required
Promise.allSettled()	ALL complete	NEVER	Partial results OK
Promise.race()	FIRST settles	First to settle rejects	Timeout pattern
Promise.any()	FIRST succeeds	ALL fail	Redundancy / fallback

5. async/await — The Modern Way

async/await is syntactic sugar over Promises. Makes async code look synchronous.

5.1 The Basics

```
// async function ALWAYS returns a Promise
async function greet() { return "Hello"; }
// Same as: function greet() { return Promise.resolve("Hello"); }
// await PAUSES the function until Promise resolves
async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  // Pauses here ^^^ but Event Loop continues!
  const data = await response.json();
  // Pauses again ^^^
  return data;
}
```

5.2 What await Actually Does Internally

```
// This async/await code:
async function example() {
  console.log("A");           // SYNC
  const data = await fetchData();
  console.log("B", data);     // MICROTASK (after await)
}

// Is equivalent to this Promise code:
function example() {
  console.log("A");           // SYNC
  return fetchData().then(data => {
    console.log("B", data);   // MICROTASK
  });
}

// Before first await = SYNC. After await = MICROTASK.
```

5.3 Error Handling with try/catch

```
async function loadUsers() {
  try {
    const res = await fetch("/api/users");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();
    return await processData(data);
  } catch (error) {
    // Catches: network errors, HTTP errors, JSON parse,
    // processData errors – ANY error from ANY await
    console.error("Failed:", error.message);
    return []; // fallback
  } finally {
    hideLoadingSpinner(); // always runs
  }
}
```

5.4 Sequential vs Parallel — Critical Difference!

```
// SEQUENTIAL – each waits for previous (SLOW!)
async function loadDashboardSlow() {
  const users = await fetch("/users").then(r => r.json());
  // waits 2s...
  const posts = await fetch("/posts").then(r => r.json());
  // waits 1s...
  const comments = await fetch("/comments").then(r => r.json());
  // waits 0.5s... Total: 3.5s!
}

// PARALLEL – all start at once (FAST!)
async function loadDashboardFast() {
```

```

const [users, posts, comments] = await Promise.all([
  fetch("/users").then(r => r.json()),
  fetch("/posts").then(r => r.json()),
  fetch("/comments").then(r => r.json()),
]);
// Total: 2s (slowest request)!
}
// PARALLEL with individual error handling
async function loadDashboardSafe() {
  const results = await Promise.allSettled([
    fetch("/users").then(r => r.json()),
    fetch("/posts").then(r => r.json()),
    fetch("/comments").then(r => r.json()),
  ]);
  return {
    users: results[0].status === "fulfilled" ? results[0].value : [],
    posts: results[1].status === "fulfilled" ? results[1].value : [],
  };
}

```

6. Real-World Production Patterns

6.1 Retry with Exponential Backoff

```

async function fetchWithRetry(url, opts = {}, maxRetries = 3) {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    try {
      const res = await fetch(url, opts);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return await res.json();
    } catch (error) {
      if (attempt === maxRetries) {
        throw new Error(`Failed after ${maxRetries+1} attempts`);
      }
      const delay = Math.pow(2, attempt) * 1000; // 1s, 2s, 4s
      await new Promise(r => setTimeout(r, delay));
    }
  }
}

```

6.2 Concurrent with Limit (Batch Processing)

```

async function processWithLimit(items, fn, concurrency = 5) {
  const results = [];
  let index = 0;
  async function worker() {
    while (index < items.length) {
      const i = index++;
      results[i] = await fn(items[i]);
    }
  }
  const workers = Array.from(
    { length: Math.min(concurrency, items.length) },
    () => worker()
  );
  await Promise.all(workers);
  return results;
}
// Upload 100 images, max 5 concurrent
const urls = await processWithLimit(images, uploadImage, 5);

```

6.3 AbortController — Cancel Async Operations

```
const controller = new AbortController();
async function searchUsers(query) {
  try {
    const res = await fetch(`/api/search?q=${query}`, {
      signal: controller.signal,
    });
    return await res.json();
  } catch (error) {
    if (error.name === "AbortError") return null; // cancelled
    throw error;
  }
}
// Cancel on timeout or user action
setTimeout(() => controller.abort(), 5000);
cancelBtn.addEventListener("click", () => controller.abort());
```

React Pattern: Cancel on Unmount

```
function SearchResults({ query }) {
  const [results, setResults] = useState([]);
  useEffect(() => {
    const controller = new AbortController();
    async function search() {
      try {
        const res = await fetch(`/api/search?q=${query}`, {
          signal: controller.signal,
        });
        setResults(await res.json());
      } catch (e) {
        if (e.name !== "AbortError") console.error(e);
      }
    }
    search();
    return () => controller.abort(); // cleanup on unmount!
  }, [query]);
}
```

6.4 Async Generators (Stream Processing)

```
async function* fetchPages(baseUrl) {
  let page = 1, hasMore = true;
  while (hasMore) {
    const res = await fetch(`${baseUrl}?page=${page}`);
    const data = await res.json();
    yield data.items; // yield this page
    hasMore = data.hasNextPage;
    page++;
  }
}
// Consume with for-await-of
const allUsers = [];
for await (const pageUsers of fetchPages("/api/users")) {
  allUsers.push(...pageUsers);
}
```

7. Next.js / MERN Patterns

7.1 Server Actions (Next.js 14+)


```

// app/actions.js
"use server";
export async function createUser(formData) {
  try {
    const user = await prisma.user.create({
      data: {
        name: formData.get("name"),
        email: formData.get("email"),
      },
    });
    revalidatePath("/users");
    return { success: true, user };
  } catch (error) {
    return { success: false, error: error.message };
  }
}

// app/components/UserForm.jsx
"use client";
function UserForm() {
  const [isPending, startTransition] = useTransition();
  async function handleSubmit(formData) {
    startTransition(async () => {
      const result = await createUser(formData);
      result.success ? toast.success("Created!") : toast.error(result.error);
    });
  }
  return (
    <form action={handleSubmit}>
      <input name="name" required />
      <button disabled={isPending}>{isPending ? "...": "Create"}</button>
    </form>
  );
}

```

7.2 API Route with Prisma + Error Handling

```

// app/api/users/route.js
export async function GET(request) {
  try {
    const { searchParams } = new URL(request.url);
    const page = parseInt(searchParams.get("page") ?? "1");
    const limit = parseInt(searchParams.get("limit") ?? "10");
    const [users, total] = await Promise.all([
      prisma.user.findMany({
        skip: (page - 1) * limit,
        take: limit,
        orderBy: { createdAt: "desc" },
      }),
      prisma.user.count(),
    ]);
    return Response.json({
      data: users,
      meta: { page, limit, total, pages: Math.ceil(total/limit) },
    });
  } catch (error) {
    if (error.code === "P2002")
      return Response.json({ error: "Duplicate" }, { status: 409 });
    return Response.json({ error: "Server error" }, { status: 500 });
  }
}

```

7.3 Custom useAsync Hook

```
function useAsync(asyncFn, deps = []) {
  const [state, setState] = useState({
    data: null, error: null, loading: true,
  });
  useEffect(() => {
    const controller = new AbortController();
    let cancelled = false;
    setState(prev => ({ ...prev, loading: true, error: null }));
    asyncFn(controller.signal)
      .then(data => {
        if (!cancelled) setState({ data, error: null, loading: false });
      })
      .catch(error => {
        if (!cancelled && error.name !== "AbortError")
          setState({ data: null, error, loading: false });
      });
    return () => { cancelled = true; controller.abort(); };
  }, deps);
  return state;
}

// Usage
function UserProfile({ userId }) {
  const { data, error, loading } = useAsync(
    (signal) => fetch(`/api/users/${userId}`, { signal })
      .then(r => r.json()),
    [userId]
  );
  if (loading) return <Spinner />;
  if (error) return <Error message={error.message} />;
  return <Profile user={data} />;
}
```

8. Gotchas & Common Mistakes

Gotcha 1: Forgetting to await

```
// BAD: gets Promise object, not the value!
async function getUser() {
  const response = fetch("/api/user"); // missing await!
  const data = response.json();        // TypeError!
}

// FIX: always await async operations
const response = await fetch("/api/user");
const data = await response.json();
```

Gotcha 2: await in forEach (BROKEN!)

```
// BAD: forEach doesn't wait for async!
items.forEach(async (item) => {
  await processItem(item); // fires simultaneously!
});
console.log("Done!"); // prints BEFORE items processed!

// FIX: for...of (sequential)
for (const item of items) { await processItem(item); }
// FIX: Promise.all + map (parallel)
await Promise.all(items.map(item => processItem(item)));
```

Gotcha 3: Error swallowed in Promise.all

```
// BAD: one failure loses ALL results
```

```
const [a, b, c] = await Promise.all([fetchA(), fetchB(), fetchC()]);
// If fetchB fails, a and c are lost too!
// FIX: use allSettled for partial results
const results = await Promise.allSettled([fetchA(), fetchB(), fetchC()]);
const successes = results.filter(r => r.status === "fulfilled");
```

Gotcha 4: Async Constructor (Not Allowed!)

```
// BAD: constructors can't be async
class Database {
  async constructor(uri) { /* SyntaxError! */ }
}
// FIX: static factory method
class Database {
  constructor(conn) { this.conn = conn; }
  static async create(uri) {
    const conn = await connect(uri);
    return new Database(conn);
  }
}
const db = await Database.create("mongodb://localhost");
```

Gotcha 5: Stale Closures with async/await (React)

```
function Counter() {
  const [count, setCount] = useState(0);
  async function handleClick() {
    setCount(count + 1); // captures count = 0
    await delay(1000);
    setCount(count + 1); // STILL captures count = 0!
    // Result: count = 1, NOT 2!
  }
  // FIX: functional updates
  async function handleClickFixed() {
    setCount(prev => prev + 1); // always latest
    await delay(1000);
    setCount(prev => prev + 1); // always latest. Result: 2!
  }
}
```

Gotcha 6: Floating Promises (Unhandled Errors)

```
// BAD: no await, no .catch() -> error silently lost
async function doWork() {
  riskyOperation(); // returns Promise, we ignore it!
}
// FIX: await it, or add .catch()
await riskyOperation();
riskyOperation().catch(err => console.error(err));
```

9. Complete Comparison

Feature	Callbacks	Promises	async/await
Syntax	fn(args, callback)	.then().catch()	await / try-catch
Error handling	Manual each level	.catch() chains	try/catch blocks
Nesting	Callback hell	Flat chains	Flat (sync-looking)
Parallel	Manual logic	Promise.all()	Promise.all()

Cancellation	Custom logic	AbortController	AbortController
Return value	Nothing	Promise	Promise
Debug	Hard (broken stack)	Better	Best (sync-like trace)
Queue type	Task Queue	Microtask Queue	Microtask Queue

10. Cheat Sheet

```
// CREATING PROMISES:
new Promise((resolve, reject) => { ... })
Promise.resolve(value)
Promise.reject(error)
// CONSUMING:
promise.then(onSuccess).catch(onError).finally(onDone)
// COMBINATORS:
Promise.all([...])           // all succeed or first fails
Promise.allSettled([...])    // all complete, every result
Promise.race([...])          // first to settle wins
Promise.any([...])           // first success wins
// ASYNC/AWAIT:
async function fn() { }      // always returns Promise
const x = await promise      // pauses until resolved
// PARALLEL: const [a,b] = await Promise.all([fnA(), fnB()]);
// SEQUENTIAL: for (const i of items) { await process(i); }
// NEVER: items.forEach(async (i) => { await ... }) // BROKEN!
```

✓ Golden Rules

1. async function always returns a Promise
2. Before first await = sync, after = microtask
3. Use `Promise.all()` for parallel, `for...of` for sequential
4. NEVER use `await` inside `forEach`
5. Always handle errors: `try/catch` or `.catch()`
6. Use `AbortController` to cancel requests on unmount
7. Use functional state updates after `await` in React
8. Use `Promise.allSettled()` when partial results are acceptable

★ Interview One-Liner

"Promises represent a future value and solve callback hell with flat chains and centralized error handling. `async/await` is syntactic sugar over Promises — code before `await` runs synchronously, code after goes to the Microtask Queue. For parallel execution, use `Promise.all()`; for sequential, use `for...of`. Never use `await` in `forEach` — it doesn't wait. Always handle rejections and cancel pending requests on component unmount with `AbortController`."